Indice

runtime-tvm, Architecture, Images and Configuration

> **Document updated to the current architecture.** It supersedes earlier versions that described compile as a Kaniko multistage build and serving with a per-model image. Today **compile is a single Job** running `compiler.py`, and **serving is model-centric** (an init container downloads the `.so`, and a generic, swappable serve image, Rust or native Go, exposes it).

The DigitalHub TVM system integrates Apache TVM as Kubernetes infrastructure and is made of **four projects**:

Project	Role	Produces
digitalhub-core (`runtimes/runtime-tvm`)	Java/Spring integration: registers the runtime, builds the K8s Jobs/Deployment, chains the phases	the three tasks `tvm+build` / `tvm+compile` / `tvm+serve`
digitalhub-tvm-toolkit	image with TVM plus the Python build/compile scripts	image `tvm-toolkit` (used by build and compile)
digitalhub-tvm-rust	native serving server in Rust	image `tvm-runtime-rust` (serve)
digitalhub-serverless	native Nuclio runtime in Go (cgo to TVM)	image `tvm-runtime-go` (serve, alternative to rust)

Pipeline:

```
function (kind: tvm)                                                  Models produced
   |  spec.model = ONNX source
   v
tvm+build    -- Job (tvm-toolkit image, builder_onnx.py)  ------->  Model  tvm-ir  (Relax IR)
   |              writes function.spec.ir_model
   v
tvm+compile  -- Job (tvm-toolkit image, compiler.py) ------------>  Model  tvm-so  (model.so)
   |              writes function.spec.so_model
   v
tvm+serve    -- Deployment (init container downloads the .so --->   OpenInference v2 endpoint
                  + generic rust/go serve image)                    REST :8080 / gRPC :9000
```

Table of contents

1. How the TVM runtime works today

The DigitalHub TVM runtime integrates Apache TVM as a Kubernetes infrastructure with three sequential tasks: **tvm+build** transforms ONNX models into Relax IR, **tvm+compile** lowers the IR to a compiled binary `model.so` for specific hardware, and **tvm+serve** deploys the compiled model behind a native server. The system is **model-centric**: no per-model image, an init container downloads the `.so` and a generic service image exposes it via OpenInference v2 on HTTP (8080) and gRPC (9000).

Registration and composition of specs

The runtime registers with `@RuntimeComponent(runtime = "tvm")` and handles three **Kind** execution types: `tvm+build`, `tvm+compile`, `tvm+serve`. Each type is governed by a triple of specs:

Type	Function Spec	Task Spec	Run Spec
tvm+build	`TvmFunctionSpec`	`TvmBuildTaskSpec`	`TvmBuildRunSpec`
tvm+compile	`TvmFunctionSpec`	`TvmCompileTaskSpec`	`TvmCompileRunSpec`
tvm+serve	`TvmFunctionSpec`	`TvmServeTaskSpec`	`TvmServeRunSpec`

The `TvmFunctionSpec` declares the source model (`model`, `format`) and hosts the two derived artifacts that the runtime writes automatically: - `ir_model`: store:// key of the Relax IR built by tvm+build - `so_model`: store:// key of the model.so compiled by tvm+compile

Task specs (`TvmBuildTaskSpec` etc.) inherit from `K8sFunctionTaskBaseSpec` and carry phase parameters (builder image, compilation options, replica count).

Each **run spec** (`TvmBuildRunSpec` etc.) aggregates `@JsonUnwrapped` both the function spec and task spec into a single flat document; the runtime composes them in the `build()` method based on precedence: run spec → fills empty fields from task spec → function spec overwrites everything. The `TvmRuntime` has three lifecycle managers annotated with `@RuntimeComponent(runtime = "tvm+build|compile|serve")` to register each type in the Kubernetes state machine.

Pipeline: frontend dispatch, builder, compiler, serve

tvm+build: source format to Relax IR

`TvmBuildRunner` receives the source ONNX model and routes it to the ONNX frontend:

1. **Format resolution**: if `function.spec.format` is explicit (not "auto"), it uses it; otherwise scans registered frontends to recognize the file extension. Raises an error if it cannot auto-detect (e.g., store:// without extension: requires explicit `format`).
2. **Model resolution**: converts store:// keys into concrete S3 paths (the K8s init container does not understand the store:// protocol).
3. **Frontend dispatch**: the ONNX frontend implements `TvmFrontend` and builds a `K8sJobRunnable` that executes the ONNX Python builder.

The builder `builder_onnx.py` receives the source model via init container (downloaded to `$TVM_INPUT_DIR/model.onnx`), converts it to Relax IR and writes: - `model.relax.json` (canonical, round-trip-safe) plus `model.relax.ir` (Relax IR text dump) - `metadata.json` (entry point, tensor specs for inputs/outputs) - `params.bin` (optional, weights if `keep_params_in_input=true`)

Then calls `_dh_publish.py` (DigitalHub SDK) to create a Model entity (type tvm-ir) and register the key in `run.status.outputs.ir_module`.

tvm+compile: Relax IR to native model.so

`TvmCompileRunner` builds a **single K8s Job** (not a multistage Kaniko build) that:

1. **Resolves the IR model**: takes explicit `task.model_path` or `function.spec.ir_model` (written by tvm+build). Raises an error if neither is present.
2. **Downloads the IR**: K8s init container downloads the S3 folder (model.relax.json + metadata.json + params.bin) to `$TVM_INPUT_DIR/`.
3. **Executes compiler.py**: injects the Python compiler binary into the `/shared/` folder, executes `/bin/bash /shared/entrypoint.sh` which dispatches to `compiler.py` with: - `--target`: hardware architecture (default llvm for CPU; supports any TVM target string) - `--opt-level` (default 3) - `--exec-mode` (bytecode or compiled) - `--relax-pipeline`, `--tir-pipeline`, `--cross-cc`, `--system-lib`, `--tag` (optional)
4. **Weight binding**: if the model was compiled with `keep_params_in_input=true`, the compiler has `params.bin` with weights as N named parameters of the entry function. `compiler.py` reads this key, uses a TVM `BindParams` transformation to convert weights into function constants, so the served model accepts only real inputs.
5. **Publishes the `.so`**: `compiler.py` calls `_dh_publish.py` to create a Model (type tvm-so), write `model.so + metadata.json` to S3 and register the key in `run.status.outputs.compiled_so`.

tvm+serve: model-centric serving

`TvmServeRunner` builds a **Kubernetes Deployment** (not a Job) that implements the model-centric paradigm:

1. **Resolves the `.so` model**: takes explicit `task.model_path` or `function.spec.so_model` (written by tvm+compile). Raises an error if absent.
2. **Init container**: downloads the S3 folder (model.so + metadata.json) to `$TVM_MODEL_DIR = /shared/model/`.
3. **Generic agnostic pod**: the serve image (configurable, default Rust tvm-runtime-rust) is launched with: - `TVM_MODEL_DIR=/shared/model` - `TVM_MODEL_NAME=<function_name>` (controllable via `task.served_name`) - HTTP port 8080 (OpenInference v2 REST) - gRPC port 9000 (GRPCInferenceService)

The image ENTRYPOINT (not defined by the runtime) reads the `.so` and `metadata.json` and exposes the model.

1. **K8s services**: the framework automatically creates services; the runner registers optional aliases: - `<function_name>-<service_name>` if declared in task spec - `<function_name>-latest` if the run belongs to the current "latest" function

Kubernetes execution framework: TvmBaseRunner and applyCommon

All runners (build/compile/serve) inherit from `TvmBaseRunner` which provides common methods:

- **Pod identity resolution**: uid/gid (default 1000:1000), home dir (default `/shared/`)
- **Entrypoint script loading**: reads `classpath:/runtime-tvm/docker/entrypoint.sh` from TvmProperties or classpath resource
- **Standard environment variables**: PROJECT_NAME, RUN_ID, TVM_HOME_DIR, TVM_INPUT_DIR, TVM_OUTPUT_DIR, TVM_OUTPUT_S3_BUCKET
- **Shared volumes**: K8s emptyDir sized by task (disk request) or configured default

The `applyCommon()` method **applies shared configuration to every runnable**, regardless of type (Job for build/compile, Deployment for serve):

```
applyCommon(
  runnable,
  run,
  taskKind,           // "tvm+build", "tvm+compile", "tvm+serve"
  funcName,
  image,
  envs,               // TVM_* variables + task-declared
  secrets,
  volumes,
  contextRefs,        // Model init container references
  taskSpec
)
```

sets on every runnable: - `runtime = "tvm"`, `task = <kind>`, `state = READY` - Label `function=<funcName>` (the framework queries it) - Image, env, secrets, volumes - CPU/memory resource from task spec, security context (uid/gid/fsGroup) - ContextRef for S3 download init container (the integrated K8s framework interprets store:// to init container)

Auto-chaining: writeModelKeyBack

When a phase completes (tvm+build or tvm+compile), `TvmRuntime.onComplete()` reads the Model key from `run.status.outputs` and writes it automatically to the function spec:

```
build: outputs.ir_module → function.spec.ir_model
compile: outputs.compiled_so → function.spec.so_model
```

The framework then calls `functionService.updateFunction()` to persist it. Thus **tvm+compile does not need to know which IR model is**: it finds it automatically on `function.spec.ir_model` written by build. Same for serve which reads `function.spec.so_model` written by compile. This **auto-chaining** means that the user executes the three tasks in order without passing manual model paths; the runtime connects them.

Two service backends

The runtime supports two interchangeable serve images (selectable via `runtime.tvm.serve`):

Rust: tvm-runtime-rust

Independent project (`digitalhub-tvm-rust`), separate build. The `tvm-serve` binary is written in pure Rust, integrates Apache TVM via C ABI (ffi), and drives the Relax VM directly. Internal architecture:

- Two async servers (axum REST + tonic gRPC) listen for requests on 8080 and 9000

- Both push inference jobs onto a shared queue (mpsc channel) drained by a pool of **N worker threads** (`TVM_SERVE_WORKERS` , default 1)
- Each worker pins itself to an OS thread with `LockOSThread()` and loads its **own copy** of the model. The Relax VM is not thread-safe, so workers cannot share one instance; running N of them gives up to N concurrent inferences at the cost of N model copies in memory
- OpenInference v2 wire format for REST, GRPCInferenceService for gRPC
- CPU only; native dtypes (FP32/FP64, INT8/16/32/64, UINT8/16/32/64). FP16 is deferred (needs an unsafe half path) and is rejected with a clear error

The build.rs files force the linker to include `libtvm_runtime.so` with `--no-as-needed` (otherwise the VM loader would not be registered statically).

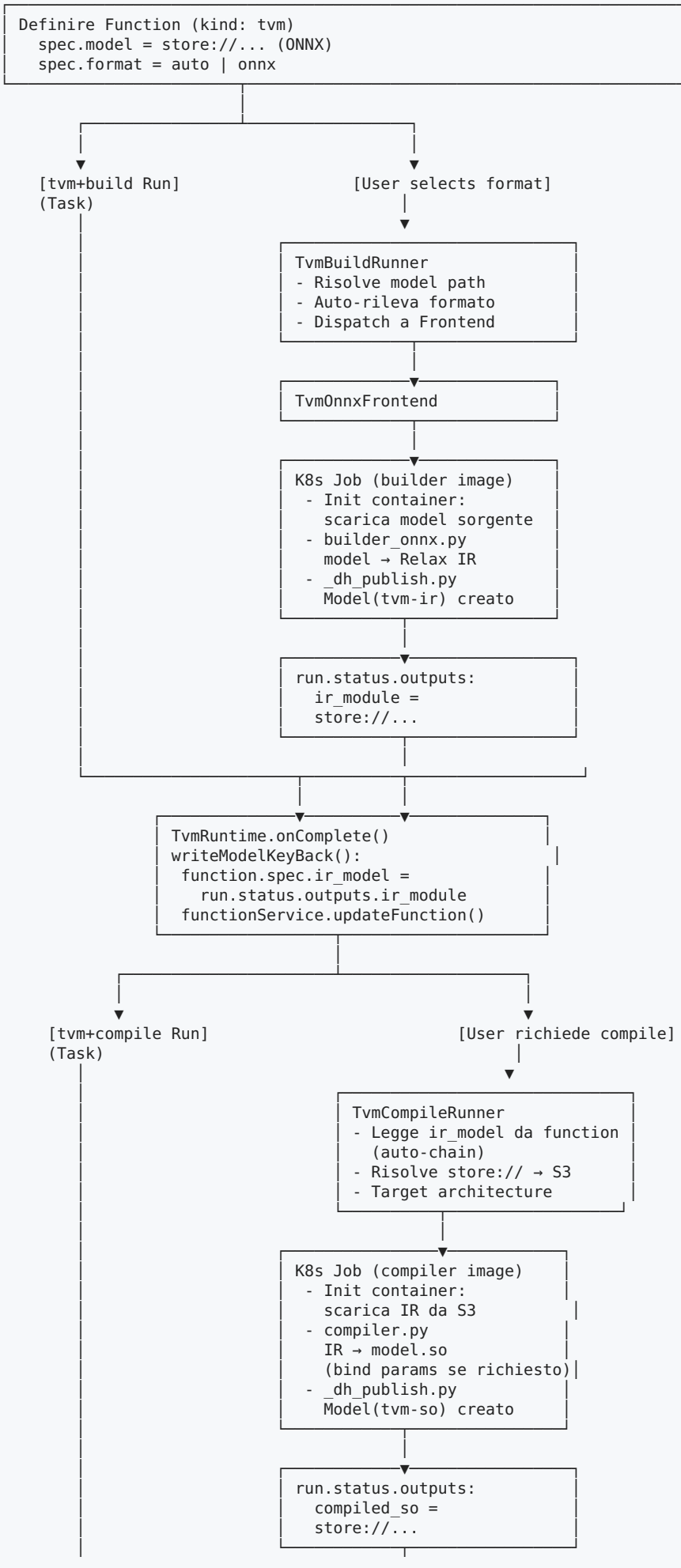
Go: native Nuclio

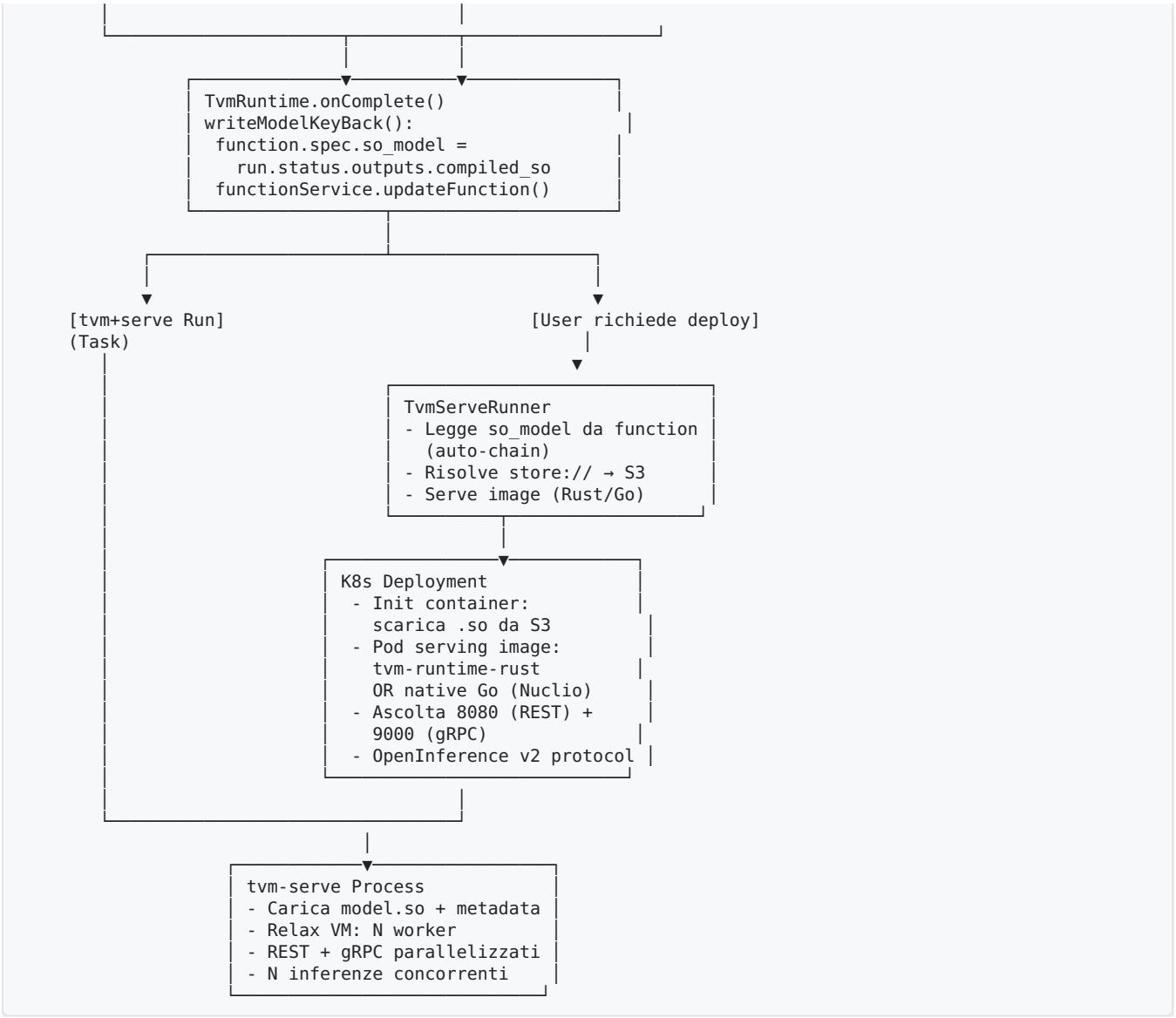
Part of **`digitalhub-serverless`** (`pkg/processor/runtime/tvm`). It is a Nuclio Go runtime that mirrors the multi-worker model of the Rust server:

- Nuclio spins up `numWorkers` workers (wired from `TVM_SERVE_WORKERS` , default 1), each with its **own in-process runtime** that loads its own copy of the model (`tvmrelax.RelaxModel`) and pins itself with `runtime.LockOSThread()`
- `ProcessEvent()` dispatches each request to a free worker, so up to N inferences run concurrently
- FFI to `libtvm_runtime.so` via cgo (package `tvmrelax`)
- Supports Nuclio OpenInference trigger (wire format v2)

Because the Relax VM and all tvm-ffi handles are `!Send/!Sync` , each worker's model instance is confined to its own OS thread; concurrency comes from running several such workers side by side rather than sharing one model. Rust build.rs files and Go go.mod replicate the `--no-as-needed` linker setup.

Flow diagram


Definire Function (kind: tvm)
spec.model = store://... (ONNX)
spec.format = auto | onnx
[tvm+build Run]
(Task)
[User selects format]
TvmBuildRunner
- Risolve model path
- Auto-rileva formato
- Dispatch a Frontend
TvmOnnxFrontend
K8s Job (builder image)
- Init container:
scarica model sorgente
- builder_onnx.py
model → Relax IR
- _dh_publish.py
Model(tvm-ir) creato
run.status.outputs:
ir_module =
store://...
TvmRuntime.onComplete()
writeModelKeyBack():
function.spec.ir_model =
run.status.outputs.ir_module
functionService.updateFunction()
[tvm+compile Run]
(Task)
[User richiede compile]
TvmCompileRunner
- Legge ir_model da function
(auto-chain)
- Risolve store:// → S3
- Target architecture
K8s Job (compiler image)
- Init container:
scarica IR da S3
- compiler.py
IR → model.so
(bind params se richiesto)
- _dh_publish.py
Model(tvm-so) creato
run.status.outputs:
compiled_so =
store://...



Runtime configuration (TvmProperties)

Property	Default	Purpose
`userId` / `groupId`	1000 / 1000	Pod identity UID/GID
`homeDir`	`/shared`	Pod working directory
`volumeSize`	(configured)	K8s emptyDir size
`bucket`	`digitalhub`	S3 bucket for artifacts
`builders`	Map	Builder image for ONNX
`compiler`	(configured)	Image that runs compiler.py
`serve`	(configured, default Rust)	Swappable serve image
`entrypoint`	classpath resource	entrypoint.sh script injected into pod
`builderScripts`	Map	Builder Python script per format

Task-level overrides (`task.image` , `task.resources` , etc.) take precedence over configured defaults.

Communication contract

All injected Python scripts (builder_*.py, compiler.py) communicate with the runtime via:

1. **Environment variables** (set by TvmBaseRunner): TVM_HOME_DIR, TVM_INPUT_DIR, TVM_OUTPUT_DIR, TVM_OUTPUT_S3_PREFIX, TVM_TASK_KIND, TVM_FUNCTION_NAME, TVM_ALGORITHM, etc.
2. **File system**: download input from INPUT_DIR (via K8s init container), write output to OUTPUT_DIR
3. **DigitalHub SDK** (`_dh_publish.py`): calls SDK to create Model entity, S3 upload, write key to `run.status.outputs` via REST PATCH

This separation allows pods to be completely generic; the runtime orchestrates the contract through environment and injected scripts.

---

2. Building the images (tvm-toolkit, tvm-rust, serverless-go)

The TVM runtime in digitalhub-core uses three specialized Docker images: one toolkit for building and compilation, and two alternative runtimes (Rust and native Go) for serving. All images are based on the TVM version locally compiled via `build-tvm.sh`.

Common Prerequisite: Compiling TVM from Source

Each image build requires a working TVM build in the local path. First, compile TVM once:

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit
./build-tvm.sh
```

This downloads Apache TVM from the official GitHub (default: the latest stable version available locally, or TVM_VERSION=0.25.0 build-tvm.sh for an explicit version), runs cmake + ninja, and deposits the build in `$TVM_BUILD/lib/` (default: `$TVM_ROOT/tvm-X.Y.Z/build/`). The script is idempotent. Re-run with FORCE=1 to recompile.

Variable	Default	Note
`TVM_VERSION`	Highest stable version compiled locally, or the remote one if not compiled	Specify with e.g. `0.25.0`
`TVM_ROOT`	`$HOME/tvm/src`	Root where TVM is cloned and compiled
`FORCE`	(not set)	Set to `1` to force recompilation

System Prerequisites (e.g. Ubuntu): `cmake ninja-build llvm-18 libllvm18 g++ git`.

Image 1: tvm-toolkit (build + compile)

Contains the TVM compiler (libtvm_compiler.so ~1.4GB, stripped by build-image.sh) and the Python runtime, packaged from the resolved TVM build host. It serves both `tvm+build` (transforms ONNX to Relax IR) and `tvm+compile` (IR to model.so).

Prerequisites: - TVM compiled locally (see above) - Access to `.so` files and Python in `$TVM_BUILD/lib/` and `$TVM_SRC/python/`

Build:

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit

# Resolves the TVM version; packages lib/ and python/ from the build host; runs docker build
./build-image.sh
```

Control Variables:

Variable	Default	Meaning
`TVM_VERSION`	Resolved by `_tvm-version.sh` (highest version compiled)	Which TVM to package
`TVM_ROOT`	`$HOME/tvm/src`	TVM lookup root
`TAG`	`tvm-toolkit:<version>` (e.g. `tvm-toolkit:0.25`)	Local image name
`REGISTRY`	(empty)	Remote registry prefix (e.g. `ghcr.io/acme`)

Push Options:

```
# Load into minikube image for local development
./build-image.sh --load

# Push to remote registry
REGISTRY=ghcr.io/acme ./build-image.sh --push

# Combine both
TAG=tvm-toolkit:0.25 REGISTRY=ghcr.io/acme ./build-image.sh --load --push
```

Artifact produced: Docker image with: - `/opt/tvm/lib/`: libtvm_runtime.so, libtvm_ffi.so, libtvm_compiler.so (all stripped) - `/opt/tvm/python/`: the tvm Python module - Base ubuntu:24.04 with dependencies (python3, native gcc/g++ + ARM64 cross-compiler, libllvm18)

Reference in digitalhub-core:

```
# runtime-tvm.yml
builders:
  onnx:      ${RUNTIME_TVM_BUILDER_ONNX:ghcr.io/scc-digitalhub/tvm-toolkit:0.25}

compiler: ${RUNTIME_TVM_COMPILER:ghcr.io/scc-digitalhub/tvm-toolkit:0.25}
```

The ONNX builder and the K8s compilation Job use the same image (or can be overridden at the task level). Set the environment variable to the exact reference after pushing:

```
export RUNTIME_TVM_COMPILER=ghcr.io/acme/tvm-toolkit:0.25
export RUNTIME_TVM_BUILDER_ONNX=ghcr.io/acme/tvm-toolkit:0.25
```

Image 2: tvm-runtime-rust (native Rust serving)

Contains the `tvm-serve` binary (Rust compiled) and the TVM runtime (libtvm_runtime.so, libtvm_ffi.so), exposes OpenInference v2 on REST 8080 / gRPC 9000. It is model-agnostic: the .so model is downloaded to `TVM_MODEL_DIR` by a K8s init container at runtime.

Prerequisites: - TVM compiled locally - Rust 1.80+ - cargo to compile tvm-serve

Build:

```
cd ~/IdeaProjects/digitalhub-tvm-rust

# Resolves TVM_BUILD; runs cargo build --release (compiles tvm-serve);
# stages lib/ + binary; runs docker build
./build-image.sh
```

Control Variables:

Variable	Default	Meaning
`TVM_HOME`	`$HOME/tvm/src/tvm-current`	TVM source path (with symlink tvm-current → tvm-0.X.Y)
`TVM_BUILD`	`$TVM_HOME/build`	TVM build path (where .so files reside)
`TVM_TAG`	`0.25`	Version for image tag (extracted from TVM_HOME if not explicit)
`TAG`	`tvm-runtime-rust:${TVM_TAG}`	Local image name
`REGISTRY`	(empty)	Remote registry prefix

Push Options:

```
# Load into minikube
./build-image.sh --load

# Push to remote registry
REGISTRY=ghcr.io/acme ./build-image.sh --push

# Specific: version and registry
TVM_BUILD=/opt/tvm/build TAG=tvm-runtime-rust:0.25 REGISTRY=ghcr.io/acme ./build-image.sh --push
```

Detailed Build Process:

1. The Rust workspace (`Cargo.toml` root, members: `crates/tvm-relax` , `crates/tvm-serve`)
2. The script runs `cargo build --release` with `TVM_BUILD_DIR=$TVM_BUILD` (cgo linkage)
3. Copies `libtvm_runtime.so` and `libtvm_ffi.so` from `$TVM_BUILD/lib/` to the build context
4. Copies the compiled `tvm-serve` binary from Rust target
5. `docker build` assembles the image: ubuntu:24.04 base + lib + binary

Artifact produced: Docker image with: - `/opt/tvm/lib/` : libtvm_runtime.so, libtvm_ffi.so (NOT the compiler) - `/usr/local/bin/tvm-serve` : the compiled server - `ENV TVM_MODEL_DIR=/model` : folder where the init container deposits the model - `ENV TVM_SERVE_PORT=8080, TVM_SERVE_GRPC_PORT=9000` - ENTRYPOINT: `/usr/local/bin/tvm-serve`

Reference in digitalhub-core:

```
serve: ${RUNTIME_TVM_SERVE:ghcr.io/scc-digitalhub/tvm-runtime-rust:0.25}
```

After pushing, set:

```
export RUNTIME_TVM_SERVE=ghcr.io/acme/tvm-runtime-rust:0.25
```

Image 3: tvm-runtime-go (native Go serving)

Alternative native runtime: Nuclio processor compiled in Go, linked against libtvm_ffi/libtvm_runtime via cgo. Zero Python wrapper. Serves OpenInference v2 on REST/gRPC. Model-centric like Rust: init container downloads the .so model to `TVM_MODEL_DIR`.

Prerequisites: - TVM compiled locally, including FFI headers in `$TVM_HOME/3rdparty/tvm-ffi/include/` - Go 1.25+

Build:

```
cd ~/IdeaProjects/digitalhub-serverless/images/tvm

# Stages the build context (tvm-include/, tvm-lib/, src/, Dockerfile, entrypoint.sh);
# runs multistage docker build (build in golang:1.25 + runtime ubuntu:24.04)
./build.sh
```

Control Variables:

Variable	Default	Meaning
`TVM_HOME`	`$HOME/tvm/src/tvm-current`	TVM source path (with symlink tvm-current → tvm-0.X.Y)
`TVM_BUILD`	`$TVM_HOME/build`	TVM build path
`TVM_TAG`	`0.25`	Version for image tag
`TAG`	`tvm-runtime-go:${TVM_TAG}`	Local image name
`REGISTRY`	(empty)	Remote registry prefix

Push Options:

```
# Load into minikube
./build.sh --load

# Push to remote registry
REGISTRY=ghcr.io/acme ./build.sh --push

# Specific
TVM_BUILD=/opt/tvm/build TAG=tvm-runtime-go:0.25 REGISTRY=ghcr.io/acme ./build.sh --push
```

Detailed Build Process:

1. Stages in a temp context: - `tvm-include/{tvm-ffi,dlpack}/` : C headers from TVM FFI dependencies - `tvm-lib/{libtvm_ffi,libtvm_runtime}.so` : runtime libraries - `src/` : digitalhub-serverless source (excluding .git, test, images) - `Dockerfile`, `entrypoint.sh`

2. Multistage build: - **Build phase**: golang:1.25 with CGO_ENABLED=1, compiles `cmd/processor/main.go` linking cgo against libtvm_ffi/libtvm_runtime - **Runtime phase**: ubuntu:24.04 + the .so files + compiled processor binary

3. Entrypoint: script `entrypoint.sh` generates Nuclio config from env and launches the processor

Artifact produced: Docker image with: - `/opt/tvm/lib/` : libtvm_runtime.so, libtvm_ffi.so - `/opt/nuclio/processor` : compiled Nuclio binary - `ENV TVM_MODEL_DIR=/shared/model` : init container downloads the model here - `ENV TVM_MODEL_NAME=model, TVM_SERVE_PORT=8080, TVM_SERVE_GRPC_PORT=9000` - ENTRYPOINT: `/entrypoint.sh` (generates config and launches processor)

Reference in digitalhub-core:

```
serve: ${RUNTIME_TVM_SERVE:ghcr.io/scc-digitalhub/tvm-runtime-rust:0.25}
```

You can also use the Go runtime by setting:

```
export RUNTIME_TVM_SERVE=ghcr.io/acme/tvm-runtime-go:0.25
```

End-to-End Flow: Build, Push, and Configuration

Scenario: local build + remote registry (minikube → push → CORE config)

```
# 1. Compile TVM once
cd ~/IdeaProjects/digitalhub-tvm-toolkit
./build-tvm.sh  # ~20-60 min; verify with ls ~/tvm/src/tvm-0.25.0/build/lib/libtvm_*.so

# 2. Build tvm-toolkit (build + compile)
./build-image.sh --load --push \
  TAG=tvm-toolkit:0.25 REGISTRY=ghcr.io/myorg
# Result: loads into minikube AND pushes to ghcr.io/myorg/tvm-toolkit:0.25

# 3. Build tvm-runtime-rust (serve)
cd ~/IdeaProjects/digitalhub-tvm-rust
./build-image.sh --load --push \
  TAG=tvm-runtime-rust:0.25 REGISTRY=ghcr.io/myorg
# Result: loads into minikube AND pushes to ghcr.io/myorg/tvm-runtime-rust:0.25

# 4. Build tvm-runtime-go (alternative serve, optional)
cd ~/IdeaProjects/digitalhub-serverless/images/tvm
./build.sh --load --push \
  TAG=tvm-runtime-go:0.25 REGISTRY=ghcr.io/myorg
# Result: ghcr.io/myorg/tvm-runtime-go:0.25

# 5. Configure CORE to point to your images
export RUNTIME_TVM_COMPILER=ghcr.io/myorg/tvm-toolkit:0.25
export RUNTIME_TVM_BUILDER_ONNX=ghcr.io/myorg/tvm-toolkit:0.25
export RUNTIME_TVM_SERVE=ghcr.io/myorg/tvm-runtime-rust:0.25
# (or ghcr.io/myorg/tvm-runtime-go:0.25 to use the Go runtime)

# 6. Start CORE with environment variables set
# (or update runtime-tvm.yml in deployment)
```

Scenario: minikube only (local development)

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit
./build-image.sh --load           # TAG default: tvm-toolkit:0.25

cd ~/IdeaProjects/digitalhub-tvm-rust
./build-image.sh --load           # TAG default: tvm-runtime-rust:0.25

# CORE reads images from minikube if RUNTIME_TVM_* does not override
# (minikube image ls to verify they are loaded)
```

Note: The Model-Centric Architecture of Serving

Both `tvm-runtime-rust` and `tvm-runtime-go` are generic, they do not bake any model. Each K8s Deployment `tvm+serve`:

1. An **init container** (not shown here, managed by the framework) downloads the compiled .so model (output of `tvm+compile`) from S3 to `TVM_MODEL_DIR`
2. The **serve container** (tvm-runtime-rust or tvm-runtime-go) launches with that model already in place
3. The server reads `$TVM_MODEL_DIR/model.so` + `metadata.json` and starts serving

This decouples images from the specific model: you reuse the same serve image for all models and projects.

Cleanup and Troubleshooting

If the build fails:

```
# Verify that TVM is compiled
ls -la ~/tvm/src/tvm-0.25.0/build/lib/libtvm_*.so

# Force recompilation
cd ~/IdeaProjects/digitalhub-tvm-toolkit
FORCE=1 ./build-tvm.sh

# Retry docker build with debug
./build-image.sh --load
```

If images do not load into minikube:

```
minikube image ls | grep tvm-toolkit
# If missing, verify minikube mount with `minikube docker-env`
eval $(minikube docker-env)
./build-image.sh --load
```

If push to registry fails:

```
# Verify registry access
docker login ghcr.io

# Retry with explicit registry
REGISTRY=ghcr.io/myorg ./build-image.sh --push
docker push ghcr.io/myorg/tvm-toolkit:0.25  # manual for debug
```

---

3. Starting and configuring

Common Prerequisites

The environment requires:

Component	Version/Notes
Java	**JDK 21 minimum** (run.sh/build.sh verify it). The sandbox default is JDK 25 which silently breaks Lombok, explicitly export `JAVA_HOME` to a 21.x version before `./run.sh`
Maven	with `mvnw` included in the repo
Git	(verify the `.git` of the working directory)

Recipe 1: Local environment on minikube

Complete workflow for development: TVM runtime built locally, images in local registry, CORE running on laptop against the minikube cluster.

Phase A: Infrastructure setup

Start minikube and the local registry:

```
# Start minikube with required resources
minikube start --cpus=4 --memory=8192 --driver=docker

# Retrieve the IP of the minikube docker-network gateway
# (needed because pods must reach the registry as <gateway>:5000)
CLUSTER_GATEWAY=$(docker network inspect minikube --format '{{range .IPAM.Config}}{{.Gateway}}{{end}}')
echo "Gateway del cluster: $CLUSTER_GATEWAY"

# Launch a local registry on docker-network (reachable as 172.17.0.1:5000 from pods)
docker run -d --network minikube --name registry.local -p 5000:5000 registry:2

# Verification:
curl http://localhost:5000/v2/_catalog
```

Launch MinIO:

```
# MinIO exposed on nodeport 30900 (reachable from local CORE as <minikube_ip>:30900)
minikube kubectl -- apply -f - <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: minio
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-pvc
  namespace: minio
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minio
  namespace: minio
spec:
  replicas: 1
  selector:
    matchLabels:
      app: minio
  template:
    metadata:
      labels:
        app: minio
    spec:
      containers:
      - name: minio
        image: minio/minio:latest
        args:
          - server
          - /data
          - --console-address
          - :9001
        env:
          - name: MINIO_ROOT_USER
            value: minio
          - name: MINIO_ROOT_PASSWORD
            value: minio123
        ports:
          - containerPort: 9000
          - containerPort: 9001
        volumeMounts:
          - name: storage
            mountPath: /data
      volumes:
        - name: storage
          persistentVolumeClaim:
            claimName: minio-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: minio
  namespace: minio
spec:
  type: NodePort
  ports:
    - name: api
      port: 9000
      targetPort: 9000
      nodePort: 30900
    - name: console
      port: 9001
      targetPort: 9001
      nodePort: 30901
  selector:
    app: minio
EOF

# Verification (replace <minikube-ip> with the output of `minikube ip`)
```

```
MINIO_IP=$(minikube ip)
curl -s http://$MINIO_IP:30900/minio/health/live -o /dev/null -w '%{http_code}\n'

# Create the bucket via console (http://$MINIO_IP:30901, admin/admin123)
# Or via CLI:
docker run -it --rm --network host minio/mc:latest \
  mc mb --insecure minio/digitalhub \
  --with-lock \
  -u minio -p minio123 \
  http://$(minikube ip):30900
```

Phase B: Building TVM images

There is no image-rebuild script in the repo. The image-build process (build toolkit, cargo tvm-serve, runtime base, tag and push) must be run manually or via CI/CD. For now, use the public images (see below) or pre-pull them in minikube.

Quick reference: images are defined as environment variables and used by Java runners:

```
# runtime-tvm.yml (defaults)
builders:
  onnx:      ${RUNTIME_TVM_BUILDER_ONNX:ghcr.io/scc-digitalhub/tvm-toolkit:0.25}
compiler:    ${RUNTIME_TVM_COMPILER:ghcr.io/scc-digitalhub/tvm-toolkit:0.25}
serve:       ${RUNTIME_TVM_SERVE:ghcr.io/scc-digitalhub/tvm-runtime-rust:0.25}
```

Phase C: Environment variables and launching CORE

```
# 1. JAVA_HOME to JDK 21 (NOT JDK 25)
export JAVA_HOME=~/.sdkman/candidates/java/21.0.4-graal
# or verify with `java -version` and find the path

# 2. S3 / MinIO
MINIO_IP=$(minikube ip)
export S3_CREDENTIALS_PROVIDER=true
export S3_ENDPOINT_URL=http://$MINIO_IP:30900
export S3_BUCKET=digitalhub
export AWS_ACCESS_KEY=minio
export AWS_SECRET_KEY=minio123
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio123
export FILES_DEFAULT_STORE=s3://digitalhub

# 3. Endpoint + authentication (MANDATORY to inject S3 credentials in TVM pods)
CLUSTER_GATEWAY=$(docker network inspect minikube --format '{{range .IPAM.Config}}{{.Gateway}}{{end}}')
export DH_ENDPOINT=http://$CLUSTER_GATEWAY:8080
export DH_AUTH_BASIC_USER=admin
export DH_AUTH_BASIC_PASSWORD=admin

# 4. TVM images (point to gateway for reachability from pods, optional if using public defaults)
# export RUNTIME_TVM_BUILDER_ONNX=$CLUSTER_GATEWAY:5000/tvm-toolkit:0.25
# export RUNTIME_TVM_COMPILER=$CLUSTER_GATEWAY:5000/tvm-toolkit:0.25
# export RUNTIME_TVM_SERVE=$CLUSTER_GATEWAY:5000/tvm-runtime-rust:0.25

# 5. (Optional) Registry for re-tagging / building custom serve images
# export IMAGE_REGISTRY=$CLUSTER_GATEWAY:5000
# export KANIKO_ARGS=--insecure,--skip-tls-verify,--insecure-pull,--compressed-caching=false,--snapshot-mode=redo

# 6. Build and launch CORE
./build.sh    # Maven: compile tvm modules and application
./run.sh      # Spring Boot with default profile, port 8080
```

Verify that CORE is accessible:

```
curl -u admin:admin http://localhost:8080/actuator/health
```

Recipe 2: Remote environment on online Kubernetes cluster

Workflow for production/staging: images pushed to a private registry, CORE deployed in cluster, TVM pods download dependencies from cluster-local registry.

Files to edit and environment variables

Phase	File/Action	Value/Note
Local builds	(N/A, use CI/CD)	,
Registry push	Env for `docker push`	`REGISTRY=your-registry.io/namespace`
CORE config	`application.properties` or K8s env	`spring.profiles.active=kubernetes`
K8s Deployment CORE	Helm values / YAML manifest	namespace, image, resources, env
TVM images	Cluster env (ConfigMap/ Secret)	`RUNTIME_TVM_BUILDER_*` / `RUNTIME_TVM_COMPILER` / `RUNTIME_TVM_SERVE` = private registry addresses
External S3	K8s Secret (AWS_ / *S3*_)	S3_ENDPOINT_URL, bucket, credentials
Private registry pull	K8s Secret of type `docker-registry`	Inherited from `K8S_REGISTRY_SECRET` in TVM pods

Phase A: Build and push images

```
# 1. Build (using Docker Buildx or similar)
docker build -t your-registry.io/tvm-toolkit:0.25 -f Dockerfile.toolkit .
docker build -t your-registry.io/tvm-runtime-rust:0.25 -f Dockerfile.runtime .

# 2. Push (authenticate with `docker login` first)
docker push your-registry.io/tvm-toolkit:0.25
docker push your-registry.io/tvm-runtime-rust:0.25
```

Phase B: Deploying CORE in K8s

Minimal manifest (Helm or plain YAML):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: digitalhub-core
  namespace: digitalhub
spec:
  replicas: 1
  selector:
    matchLabels:
      app: digitalhub-core
  template:
    metadata:
      labels:
        app: digitalhub-core
    spec:
      serviceAccountName: digitalhub-core
      containers:
        - name: core
          image: ghcr.io/scc-digitalhub/digitalhub-core:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          env:
            # Spring boot
            - name: SPRING_PROFILES_ACTIVE
              value: kubernetes

            # Java (JDK 21 built-in in the image)
            - name: JAVA_TOOL_OPTIONS
              value: "-Xmx2g"

            # External S3
            - name: S3_CREDENTIALS_PROVIDER
              value: "true"
            - name: S3_ENDPOINT_URL
              valueFrom:
                configMapKeyRef:
                  name: digitalhub-config
                  key: s3.endpoint_url
            - name: S3_BUCKET
              value: digitalhub
            - name: AWS_ACCESS_KEY_ID
              valueFrom:
                secretKeyRef:
                  name: digitalhub-s3
                  key: access_key
            - name: AWS_SECRET_ACCESS_KEY
              valueFrom:
                secretKeyRef:
                  name: digitalhub-s3
                  key: secret_key
            - name: FILES_DEFAULT_STORE
              value: s3://digitalhub

            # TVM images (private registry)
            - name: RUNTIME_TVM_BUILDER_ONNX
              value: your-registry.io/tvm-toolkit:0.25
            - name: RUNTIME_TVM_COMPILER
              value: your-registry.io/tvm-toolkit:0.25
            - name: RUNTIME_TVM_SERVE
              value: your-registry.io/tvm-runtime-rust:0.25

            # For pulling from private registry in TVM pods
            # (the K8s framework inherits it automatically from builders)
            - name: K8S_REGISTRY_SECRET
              value: registry-credentials

          resources:
            requests:
              memory: "2Gi"
              cpu: "1"
            limits:
              memory: "4Gi"
              cpu: "2"

      # ImagePullSecret for private registry
      imagePullSecrets:
        - name: registry-credentials
```

```yaml
---
# Secret for S3
apiVersion: v1
kind: Secret
metadata:
  name: digitalhub-s3
  namespace: digitalhub
type: Opaque
data:
  access_key: <base64 of the key>
  secret_key: <base64 of the secret key>

---
# ConfigMap for general configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: digitalhub-config
  namespace: digitalhub
data:
  s3.endpoint_url: https://s3.amazonaws.com

---
# Secret for pulling from private registry
apiVersion: v1
kind: Secret
metadata:
  name: registry-credentials
  namespace: digitalhub
type: docker-registry
data:
  .dockerconfigjson: <base64 of Docker config.json>
```

Troubleshooting

Symptom	Cause	Remedy
`run.sh` fails: "Required java version is 21"	`JAVA_HOME` not exported or version < 21	`export JAVA_HOME=<path-to-jdk21>` and verify with `java -version`
Pod compile **OOMKilled (exit 137)**	No resource request, default limit approximately 256Mi	Specify `resources: memory: 8Gi, cpu: 4` in task spec / in K8s Pod template
Pod inference very slow (approximately 39s per input)	Default CPU (100m) too low	Add `resources: cpu: 4` to serve deployment
`ImagePullBackOff` in TVM pods	Wrong image ref or pull secret not found	Verify that `RUNTIME_TVM_*` point to correct registry (not `localhost`), check that `K8S_REGISTRY_SECRET` or imagePullSecrets are configured
Builder fails: `Unable to locate credentials`	Run not authenticated, S3 credentials not injected	Launch CORE with `DH_AUTH_BASIC_USER=admin` and `DH_AUTH_BASIC_PASSWORD` and authenticate the REST client
Compile uses the wrong target or defaults unexpectedly	Task field `target_architecture` not set in the console form	Set `target_architecture: cpu` explicitly (`llvm` still works as a legacy alias). The JSON key is `target_architecture` (deliberately not `target`, which would break the console run form); the runner passes it to the pod as the `TVM_TARGET` env var
MinIO env missing and task fails	Pod does not receive S3_ENDPOINT_URL and credentials	Verify that CORE is launched with `DH_AUTH_BASIC_*=admin/admin` and that the request is authenticated, the framework injects credentials only for authenticated runs
Serve gRPC connection refused (:9000 ok but :8080 no)	Deployed serve runtime image is old (REST-only)	Redeploy with image built from current Rust source

---

4. Environment variables of the TVM projects

This section lists only the variables that are specific to the four TVM projects (runtime-tvm, tvm-toolkit, tvm-rust, serverless-go). The general CORE variables (database, authentication, endpoint, and the shared Kubernetes framework) are the platform standard and are documented with CORE itself; only the two groups the TVM module actually relies on are recalled at the end (section 4.4).

The `Example` column shows a realistic value for a live cluster that pulls from a private registry `registry.acme.com/ml` and stores Models in an S3 bucket `acme-models`. Replace those with your own coordinates.

4.1 CORE runtime-tvm (set on the CORE deployment)

These bind `runtime-tvm.yml` and drive the images and the pod identity of every TVM Job/Deployment. In a live cluster you set them as environment variables on the CORE container. The image values must be references that the cluster can pull.

Variable	Meaning	Default	Example
`RUNTIME_TVM_BUILDER_ONNX`	tvm-toolkit image for the ONNX builder	`ghcr.io/scc-digitalhub/tvm-toolkit:0.25`	`registry.acme.com/ml/tvm-toolkit:0.25`
`RUNTIME_TVM_COMPILER`	tvm-toolkit image that runs compiler.py (IR to .so)	`ghcr.io/scc-digitalhub/tvm-toolkit:0.25`	`registry.acme.com/ml/tvm-toolkit:0.25`
`RUNTIME_TVM_SERVE`	serve base image (tvm-runtime-rust or tvm-runtime-go)	`ghcr.io/scc-digitalhub/tvm-runtime-rust:0.25`	`registry.acme.com/ml/tvm-runtime-rust:0.25`
`RUNTIME_TVM_USER_ID`	runAsUser for TVM pods	value of `kubernetes.security.user`	`1000`
`RUNTIME_TVM_GROUP_ID`	runAsGroup / fsGroup for TVM pods	value of `kubernetes.security.group`	`1000`
`RUNTIME_TVM_HOME_DIR`	working/home dir inside the pod	`/shared`	`/shared`
`RUNTIME_TVM_VOLUME_SIZE`	size of the shared scratch volume	`4Gi`	`8Gi`

Per task, the console form can still override `image` (builder/compiler/serve) and resources for a single run; the variables above are the cluster-wide defaults.

4.2 Serve image runtime (read by the rust/go serve image)

The serve runner sets `TVM_MODEL_DIR` and `TVM_MODEL_NAME` on the pod automatically; the rust image also honors the two port variables. You normally never set these by hand, they are listed for completeness.

Variable	Meaning	Default	Backend	Example
`TVM_MODEL_DIR`	folder with `model.so` + `metadata.json` (filled by the init container)	`/shared/model`	rust, go	`/shared/model`
`TVM_MODEL_NAME`	model name exposed at `/v2/models/<name>`	rust: `tvm-model`, go: `model` (the runner always sets it)	rust, go	`yolo`
`TVM_SERVE_PORT`	REST (OpenInference v2) port	`8080`	rust, go	`8080`
`TVM_SERVE_GRPC_PORT`	gRPC port	`9000`	rust, go	`9000`
`TVM_SERVE_WORKERS`	number of serve workers per pod; each worker loads its own copy of the model, so >1 allows concurrent inference (at the cost of N model copies in memory). On rust it sizes the worker-thread pool; on go it sets nuclio `numWorkers`. Set by the runner from `task.workers`.	`1`	rust, go	`2`

4.3 Image build (on the build machine or CI, not in the cluster)

These configure the `build-image.sh` / `build.sh` scripts that produce and push the three images. `REGISTRY` empty means a local build only; set it to push to your registry, and the pushed ref must match the `RUNTIME_TVM_*` you set in CORE.

Common to the rust and go build scripts (the toolkit resolves the TVM tree via `TVM_ROOT` / `TVM_VERSION` instead):

Variable	Meaning	Default	Example
`TAG`	image name:tag to build	`<image>:<major.minor>` derived from the packaged TVM version	`tvm-runtime-rust:0.25`
`REGISTRY`	push target prefix; the pushed ref is `$REGISTRY/$TAG`	empty (local only)	`registry.acme.com/ml`
`TVM_HOME`	path to the locally built TVM source	`$HOME/tvm/src/tvm-current`	`/home/ci/tvm/src/tvm-0.25.0`
`TVM_BUILD`	TVM build dir holding the shared libs	`$TVM_HOME/build`	`/home/ci/tvm/src/tvm-0.25.0/build`

Extra for tvm-toolkit (`_tvm-version.sh`):

Variable	Meaning	Default	Example
`TVM_VERSION`	TVM release to package	highest stable version built under `$TVM_ROOT`	`0.25.0`
`TVM_ROOT`	root holding the built TVM sources	`$HOME/tvm/src`	`/home/ci/tvm/src`
`TVM_FFI_VERSION`	apache-tvm-ffi version (Docker build arg)	derived from TVM_VERSION	`0.1.12`
`LLVM_VERSION`	LLVM major version baked in the image	`18`	`18`
`TVM_TAG`	image tag, major.minor of `TVM_VERSION`	(derived)	`0.25`

Example, build and push all three to the private registry:

```
export REGISTRY=registry.acme.com/ml
export TVM_ROOT=/home/ci/tvm/src TVM_VERSION=0.25.0   # toolkit resolves the TVM tree from these
export TVM_HOME=/home/ci/tvm/src/tvm-0.25.0          # rust/go read the TVM tree from here
cd digitalhub-tvm-toolkit && ./build-image.sh --push     # registry.acme.com/ml/tvm-toolkit:0.25
cd ../digitalhub-tvm-rust && ./build-image.sh --push      # registry.acme.com/ml/tvm-runtime-rust:0.25
cd ../digitalhub-serverless && ./images/tvm/build.sh --push  # registry.acme.com/ml/tvm-runtime-go:0.25
```

4.4 Global CORE variables the TVM module relies on

Not TVM-exclusive, but the module cannot run without them in a live cluster. Set them once, platform-wide, on the CORE deployment. Put the secret ones in a K8s Secret.

Variable	Meaning	Example
`K8S_REGISTRY_SECRET`	imagePullSecret name injected into every pod, so TVM pods can pull from a private registry	`acme-regcred`
`S3_CREDENTIALS_PROVIDER`	enables the S3 credentials provider used by TVM pods	`true`
`S3_ENDPOINT_URL`	S3 endpoint	`https://s3.acme.com`
`S3_BUCKET`	default S3 bucket	`acme-models`
`AWS_ACCESS_KEY`	S3 access key (in a Secret)	`AKIA...`
`AWS_SECRET_KEY`	S3 secret key (in a Secret)	`wJalr...`

Example, the TVM-related env block of the CORE Deployment:

```
env:
  - name: RUNTIME_TVM_COMPILER
    value: registry.acme.com/ml/tvm-toolkit:0.25
  - name: RUNTIME_TVM_BUILDER_ONNX
    value: registry.acme.com/ml/tvm-toolkit:0.25
  - name: RUNTIME_TVM_SERVE
    value: registry.acme.com/ml/tvm-runtime-rust:0.25
  - name: K8S_REGISTRY_SECRET
    value: acme-regcred
  - name: S3_CREDENTIALS_PROVIDER
    value: "true"
  - name: S3_ENDPOINT_URL
    value: https://s3.acme.com
  - name: S3_BUCKET
    value: acme-models
  - name: AWS_ACCESS_KEY
    valueFrom: { secretKeyRef: { name: acme-s3, key: access-key } }
  - name: AWS_SECRET_KEY
    valueFrom: { secretKeyRef: { name: acme-s3, key: secret-key } }
```

---

5. Building by hand, step by step

This section describes how to build each artifact manually, starting from the TVM C++ runtime library through to the three serving images. All paths and environment variables are exact.

5.1 Build Apache TVM from source

Location: `~/IdeaProjects/digitalhub-tvm-toolkit/build-tvm.sh`

Prerequisites: - git, cmake, ninja-build - llvm-config-18 and libllvm18 (version can be overridden with `LLVM_VERSION=X`) - g++ and g++-aarch64-linux-gnu, binutils-aarch64-linux-gnu (for cross-compilation support)

On Ubuntu 24.04:

```
apt-get install -y cmake ninja-build llvm-18 libllvm18 g++ \
  g++-aarch64-linux-gnu binutils-aarch64-linux-gnu git
```

Manual build:

The script clones TVM at a specific version, initializes submodules, and compiles using CMake and Ninja. The default storage location is `$TVM_ROOT/tvm-X.Y.Z` (default `$HOME/tvm/src/tvm-X.Y.Z`).

```
#!/usr/bin/env bash
# Resolve TVM version: queries GitHub latest stable or uses locally-built highest
# version. Exports TVM_VERSION, TVM_SRC, TVM_BUILD, TVM_TAG, TVM_FFI_VERSION, LLVM_VERSION
export TVM_VERSION=0.25.0     # override default or leave empty for auto-resolve
export TVM_ROOT=$HOME/tvm/src # or your preferred TVM source root
export LLVM_VERSION=18         # must match system llvm-config-X and libllvm

TVM_SRC="$TVM_ROOT/tvm-$TVM_VERSION"
TVM_BUILD="${TVM_BUILD:-$TVM_SRC/build}"

# Clone (or skip if already present)
if [ ! -d "$TVM_SRC" ]; then
  git clone --depth 1 --branch "v$TVM_VERSION" --recurse-submodules \
    --shallow-submodules https://github.com/apache/tvm "$TVM_SRC"
fi

# Fetch submodules if already cloned
(
  cd "$TVM_SRC"
  git submodule update --init --recursive
)

# Configure and build
mkdir -p "$TVM_BUILD"
cp "$TVM_SRC/cmake/config.cmake" "$TVM_BUILD/config.cmake"
cat >> "$TVM_BUILD/config.cmake" <<'EOF'
set(USE_LLVM "llvm-config-18")
set(CMAKE_BUILD_TYPE RelWithDebInfo)
set(USE_CCACHE AUTO)
set(USE_GTEST OFF)
EOF

cd "$TVM_BUILD"
cmake "$TVM_SRC" -G Ninja
ninja -j$(nproc)
```

Expected output: The build produces libraries in `$TVM_BUILD/lib/` : - `libtvm_runtime.so` (1-2 MB): C runtime with VM executor and module loader - `libtvm_ffi.so` (1-2 MB): FFI C bindings used by python/rust/go - `libtvm_compiler.so` (1.4 GB unstripped, ~100 MB stripped): LLVM-based codegen for compilation - `python/tvm/` (Python source): used by the toolkit image

Build time: 20-60 minutes depending on machine.

Alternative: use the wrapper script

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit
TVM_VERSION=0.25.0 ./build-tvm.sh
# or, to auto-resolve the latest built version:
./build-tvm.sh
# or rebuild from scratch even if already present:
FORCE=1 ./build-tvm.sh
```

The wrapper (`build-tvm.sh`) calls `source _tvm-version.sh` to resolve TVM_VERSION, TVM_SRC, TVM_BUILD, and version-coupled deps (LLVM_VERSION, TVM_FFI_VERSION).

5.2 Build the tvm-toolkit image (Python compiler)

Location: `~/IdeaProjects/digitalhub-tvm-toolkit/`

The `tvm-toolkit` image packages the locally-built TVM libraries (with debug symbols stripped) plus Python bindings. It is used by the build Job (ONNX to Relax IR) and the compile Job (IR to `.so` binaries).

Prerequisites: - Docker daemon running - Locally-built TVM at `$TVM_SRC/build/lib/` (run section 5.1 first) - `docker` or equivalent (can use `podman build` by adapting the examples)

Manual image build:

The wrapper script (`build-image.sh`) copies the TVM libs and python directory from the host build, strips debug symbols, then builds the image. Here is the step-by-step process:

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit

# Set resolved TVM version and deps (same as build-tvm.sh)
export TVM_VERSION=0.25.0
export TVM_ROOT=$HOME/tvm/src
export LLVM_VERSION=18
export TVM_FFI_VERSION=0.1.12
TVM_SRC="$TVM_ROOT/tvm-$TVM_VERSION"
TVM_BUILD="${TVM_SRC}/build"
TVM_TAG="${TVM_VERSION%.*}"  # e.g. 0.25

# Verify libtvm_compiler.so exists (compiler is required)
[ -f "$TVM_BUILD/lib/libtvm_compiler.so" ] || {
  echo "ERROR: missing $TVM_BUILD/lib/libtvm_compiler.so"
  exit 1
}

# Stage the libraries and Python source into the docker build context
ctx="./tvm-toolkit"
rm -rf "$ctx/lib" "$ctx/python"
mkdir -p "$ctx/lib" "$ctx/python"

# Copy and strip libraries (RelWithDebInfo are huge, ~1.4GB for compiler)
for so in libtvm_runtime.so libtvm_ffi.so libtvm_compiler.so; do
  cp "$TVM_BUILD/lib/$so" "$ctx/lib/$so"
  strip --strip-debug "$ctx/lib/$so" 2>/dev/null || true
done

# Copy Python TVM module (pycache removed)
cp -r "$TVM_BUILD/../python/tvm" "$ctx/python/tvm"
find "$ctx/python" -name __pycache__ -type d -prune -exec rm -rf {} + 2>/dev/null || true

# Build the image
docker build -t "tvm-toolkit:$TVM_TAG" \
  --build-arg "TVM_FFI_VERSION=$TVM_FFI_VERSION" \
  --build-arg "LLVM_VERSION=$LLVM_VERSION" \
  "$ctx"

# Clean up staging area
rm -rf "$ctx/lib" "$ctx/python"
```

Image contents: - Base: `ubuntu:24.04` (glibc 2.39, libstdc++13) - System dependencies: libllvm18, gcc, g++, cross-toolchain (arm64), python3, pip - Stripped TVM libraries: `libtvm_runtime.so`, `libtvm_ffi.so`, `libtvm_compiler.so` in `/opt/tvm/lib/` - Python TVM module in `/opt/tvm/python/` - Python packages: apache-tvm-ffi, onnx, numpy, scipy, cloudpickle, digitalhub SDK, and build tools (pytest, tornado, etc.)

Load or push:

Load into minikube for local testing:

```
# The wrapper script does this automatically with --load
./build-image.sh --load
# Or manually:
minikube image load "tvm-toolkit:0.25"
```

Push to a Docker registry:

```
# Push to Docker Hub or a private registry
REGISTRY=ghcr.io/yourorg ./build-image.sh --push
# Or manually:
docker tag "tvm-toolkit:0.25" "ghcr.io/yourorg/tvm-toolkit:0.25"
docker push "ghcr.io/yourorg/tvm-toolkit:0.25"
```

Alternative: use the wrapper script directly

```
cd ~/IdeaProjects/digitalhub-tvm-toolkit
./build-image.sh                    # builds tvm-toolkit:0.25
./build-image.sh --load             # loads into minikube
REGISTRY=myregistry ./build-image.sh --push  # pushes to myregistry/tvm-toolkit:0.25
TAG=tvm-compiler:custom ./build-image.sh     # custom tag
```

5.3 Build the tvm-runtime-rust image (Rust server)

Location: `~/IdeaProjects/digitalhub-tvm-rust/`

The `tvm-runtime-rust` image contains a native Rust binary (`tvm-serve`) that loads a TVM model (`.so` + `metadata.json`) and serves it via OpenInference v2 (REST 8080 / gRPC 9000). The binary is compiled with `cargo build --release` linked against `libtvm_runtime.so` and `libtvm_ffi.so`.

Prerequisites: - Rust toolchain (rustc, cargo) version 1.80+ - Protobuf compiler (vendored via `protoc-bin-vendored` crate, no system protoc needed) - Locally-built TVM at `$TVM_BUILD/lib/` (run section 5.1 first) - Docker daemon

Manual build:

```
cd ~/IdeaProjects/digitalhub-tvm-rust

# Resolve TVM version
export TVM_VERSION=0.25.0
export TVM_ROOT=$HOME/tvm/src
TVM_HOME="$TVM_ROOT/tvm-$TVM_VERSION"
TVM_BUILD="${TVM_HOME}/build"
TVM_TAG="${TVM_VERSION%.*}"  # e.g. 0.25

# Verify the TVM libraries exist
[ -f "$TVM_BUILD/lib/libtvm_runtime.so" ] || {
  echo "ERROR: missing $TVM_BUILD/lib/libtvm_runtime.so"
  exit 1
}

# Compile the Rust binary
# The build script (crates/tvm-serve/build.rs) requires TVM_BUILD_DIR.
# It generates gRPC code from proto files using vendored protoc.
TVM_BUILD_DIR="$TVM_BUILD" cargo build --release --bin tvm-serve

# Verify the binary was created
[ -f "./target/release/tvm-serve" ] || {
  echo "ERROR: cargo build failed"
  exit 1
}

# Stage libraries and binary for Docker build
rm -rf "./lib"
mkdir -p "./lib"
cp "$TVM_BUILD/lib/libtvm_runtime.so" "$TVM_BUILD/lib/libtvm_ffi.so" "./lib/"
cp "./target/release/tvm-serve" "./tvm-serve"

# Build the Docker image
docker build -t "tvm-runtime-rust:$TVM_TAG" .

# Clean up staging files
rm -rf "./lib" "./tvm-serve"
```

Image contents: - Base: `ubuntu:24.04` - Runtime dependencies: libstdc++6 - Stripped TVM libraries: `libtvm_runtime.so`, `libtvm_ffi.so` in `/opt/tvm/lib/` - Rust binary: `/usr/local/bin/tvm-serve` - Entry point: `tvm-serve` (expects `TVM_MODEL_DIR=/model` to contain `model.so` + `metadata.json`) - Exposed ports: 8080 (REST), 9000 (gRPC)

Load or push:

```
# Load into minikube
./build-image.sh --load

# Or manually
minikube image load "tvm-runtime-rust:0.25"

# Push to registry
REGISTRY=ghcr.io/yourorg ./build-image.sh --push

# Or manually
docker tag "tvm-runtime-rust:0.25" "ghcr.io/yourorg/tvm-runtime-rust:0.25"
docker push "ghcr.io/yourorg/tvm-runtime-rust:0.25"
```

Alternative: use the wrapper script

```
cd ~/IdeaProjects/digitalhub-tvm-rust
./build-image.sh                    # builds tvm-runtime-rust:0.25
./build-image.sh --load             # loads into minikube
REGISTRY=myregistry ./build-image.sh --push
```

5.4 Build the tvm-runtime-go image (Go server)

Location: `~/IdeaProjects/digitalhub-serverless/images/tvm/`

The `tvm-runtime-go` image contains a Nuclio processor written in Go that links against TVM via cgo. It loads a TVM model (`.so` + `metadata.json`) and serves OpenInference v2 via a Nuclio processor runtime.

Prerequisites: - Go 1.25+ - Locally-built TVM at `$TVM_BUILD/` (run section 5.1 first) - Docker daemon - The digitalhub-serverless source tree

Manual build:

The build script stages TVM headers and libraries into a temporary Docker build context, then builds a two-stage image: first a golang:1.25 stage that compiles the processor with cgo, then a runtime stage with the binary and TVM libraries.

```
cd ~/IdeaProjects/digitalhub-serverless/images/tvm

# Resolve TVM version
export TVM_VERSION=0.25.0
export TVM_ROOT=$HOME/tvm/src
TVM_HOME="$TVM_ROOT/tvm-$TVM_VERSION"
TVM_BUILD="${TVM_HOME}/build"
FFI_INC="$TVM_HOME/3rdparty/tvm-ffi/include"
DLPACK_INC="$TVM_HOME/3rdparty/tvm-ffi/3rdparty/dlpack/include"
TVM_TAG="${TVM_VERSION%.*}"  # e.g. 0.25

# Verify the TVM libraries and headers exist
[ -f "$TVM_BUILD/lib/libtvm_ffi.so" ] || {
  echo "ERROR: missing $TVM_BUILD/lib/libtvm_ffi.so"
  exit 1
}
[ -f "$FFI_INC/tvm/ffi/c_api.h" ] || {
  echo "ERROR: missing TVM FFI headers at $FFI_INC"
  exit 1
}

# Create a temporary build context
ctx=$(mktemp -d)
trap 'rm -rf "$ctx"' EXIT

# Stage TVM headers and libraries
mkdir -p "$ctx/tvm-include/tvm-ffi" "$ctx/tvm-include/dlpack" "$ctx/tvm-lib"
cp -r "$FFI_INC/." "$ctx/tvm-include/tvm-ffi/"
cp -r "$DLPACK_INC/." "$ctx/tvm-include/dlpack/"
cp "$TVM_BUILD/lib/libtvm_ffi.so" "$TVM_BUILD/lib/libtvm_runtime.so" "$ctx/tvm-lib/"

# Copy the digitalhub-serverless source and the image files
REPO="$(cd ~/IdeaProjects/digitalhub-serverless && pwd)"
rsync -a --exclude .git --exclude test --exclude images "$REPO/" "$ctx/src/"
cp "$HERE/Dockerfile" "$HERE/entrypoint.sh" "$ctx/"

# Build the Docker image
docker build -t "tvm-runtime-go:$TVM_TAG" "$ctx"
```

Build stage details:

Inside the `golang:1.25` build stage, the processor is compiled with cgo flags:

```
FROM golang:1.25 AS build
COPY tvm-include /opt/tvm/include
COPY tvm-lib /opt/tvm/lib
WORKDIR /src
COPY src/ .
ENV CGO_ENABLED=1 \
    CGO_CFLAGS="-I/opt/tvm/include/tvm-ffi -I/opt/tvm/include/dlpack" \
    CGO_LDFLAGS="-L/opt/tvm/lib -Wl,-rpath,/opt/tvm/lib"
RUN go build -o /out/processor ./cmd/processor/main.go
```

The `CGO_CFLAGS` point to the TVM FFI headers (c_api.h, dlpack.h), and `CGO_LDFLAGS` link against `libtvm_ffi.so` and `libtvm_runtime.so`.

Runtime stage:

The runtime stage copies the compiled processor and TVM libraries, sets environment variables, and runs `entrypoint.sh`:

```
FROM ubuntu:24.04
RUN apt-get update \
    && apt-get install -y --no-install-recommends libstdc++6 ca-certificates \
    && rm -rf /var/lib/apt/lists/*
COPY --from=build /opt/tvm/lib/libtvm_runtime.so /opt/tvm/lib/libtvm_ffi.so /opt/tvm/lib/
COPY --from=build /out/processor /opt/nuclio/processor
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENV LD_LIBRARY_PATH=/opt/tvm/lib \
    TVM_MODEL_DIR=/shared/model \
    TVM_MODEL_NAME=model \
    TVM_SERVE_PORT=8080 \
    TVM_SERVE_GRPC_PORT=9000
EXPOSE 8080 9000
ENTRYPOINT ["/entrypoint.sh"]
```

Entrypoint behavior:

The `entrypoint.sh` writes a Nuclio processor config to `/tmp/processor.yaml` with model metadata, then executes the compiled processor binary:

```
#!/usr/bin/env bash
set -euo pipefail
CFG=/tmp/processor.yaml
cat > "$CFG" <<EOF
apiVersion: "nuclio.io/v1"
kind: NuclioFunction
metadata:
  name: ${TVM_MODEL_NAME:-model}
  namespace: nuclio
spec:
  runtime: tvm
  handler: tvm
  triggers:
    openinference:
      kind: openinference
      attributes:
        model_name: "${TVM_MODEL_NAME:-model}"
        enable_rest: true
        enable_grpc: true
        rest_port: ${TVM_SERVE_PORT:-8080}
        grpc_port: ${TVM_SERVE_GRPC_PORT:-9000}
      max_workers: ${TVM_SERVE_WORKERS:-1}
EOF
exec /opt/nuclio/processor --config "$CFG"
```

Image contents: - Base: `ubuntu:24.04` - Runtime dependencies: libstdc++6, ca-certificates - TVM libraries: `libtvm_runtime.so`, `libtvm_ffi.so` in `/opt/tvm/lib/` - Compiled processor: `/opt/nuclio/processor` - Entrypoint: `/entrypoint.sh` (generates processor.yaml and launches the Go binary) - Exposed ports: 8080 (REST), 9000 (gRPC)

Load or push:

```
# Load into minikube
cd ~/IdeaProjects/digitalhub-serverless/images/tvm
./build.sh --load

# Or manually
minikube image load "tvm-runtime-go:0.25"

# Push to registry
REGISTRY=ghcr.io/yourorg ./build.sh --push

# Or manually
docker tag "tvm-runtime-go:0.25" "ghcr.io/yourorg/tvm-runtime-go:0.25"
docker push "ghcr.io/yourorg/tvm-runtime-go:0.25"
```

Alternative: use the wrapper script directly

```
cd ~/IdeaProjects/digitalhub-serverless/images/tvm
./build.sh                    # builds tvm-runtime-go:0.25 (default version)
./build.sh --load             # loads into minikube
REGISTRY=myregistry ./build.sh --push
TVM_VERSION=0.25.0 ./build.sh
```

5.5 Summary table: images and CORE integration

Artifact	Image Name	Built From	Contains	RUNTIME_TVM_* Variable(s)
TVM C++ runtime	(not an image)	digitalhub-tvm-toolkit/ build-tvm.sh	`libtvm_runtime.so`, `libtvm_ffi.so`, `libtvm_compiler.so` (all in `build/lib/`)	`TVM_BUILD_DIR` (env var passed to cargo/compiler)
Compiler toolkit	`tvm-toolkit:X.Y`	digitalhub-tvm-toolkit/ build-image.sh	Ubuntu 24.04 + Python 3 + TVM (runtime, ffi, compiler) + gcc/g++ + cross-toolchain + ONNX, apache-tvm-ffi	`RUNTIME_TVM_BUILDER_ONNX`, `RUNTIME_TVM_COMPILER`
Rust server	`tvm-runtime-rust:X.Y`	digitalhub-tvm-rust/ build-image.sh	Ubuntu 24.04 + libstdc++6 + TVM runtime/ffi (.so) + tvm-serve Rust binary	`RUNTIME_TVM_SERVE`
Go server	`tvm-runtime-go:X.Y`	digitalhub-serverless/ images/tvm/ build.sh	Ubuntu 24.04 + libstdc++6 + TVM runtime/ffi (.so) + compiled Go processor (cgo) + Nuclio config generator	`RUNTIME_TVM_SERVE` (alternative to Rust)

Notes:

- All images default to `ubuntu:24.04` (glibc 2.39, libstdc++ 3.4.32) to match the host TVM build environment.
- The compiler toolkit includes `libtvm_compiler.so` (needed for ONNX build and IR compilation); the Rust and Go runtimes include only `libtvm_runtime.so` and `libtvm_ffi.so` (for loading pre-compiled models).
- Push a built image to a registry (e.g., `ghcr.io/yourorg/tvm-toolkit:0.25`) and point CORE to it by setting the corresponding `RUNTIME_TVM_*` environment variable or in `runtime-tvm.yml`.
- TVM version X.Y (major.minor) couples with LLVM version (default 18) and apache-tvm-ffi version (default 0.1.12 for 0.25.x). See `_tvm-version.sh` for mappings.

---

6. End-to-end example: from model to detections

This section walks through a complete workflow: taking an ONNX YOLOv8 model, building it to Relax IR, compiling it to a native shared library, and serving it behind an Open Inference v2 endpoint. We will then call inference and decode the detections.

6.1 Step 1: Register the function

First, create a function entity with the source ONNX model:

```yaml
kind: tvm
metadata:
  name: yolov8n
  project: my-project
spec:
  model: "s3://bucket/path/yolov8n.onnx"
  format: auto
```

The spec fields map directly to what the TVM runtime processes:

Field	Meaning	Editable
`model`	Source model URI: `s3://`, `https://`, `store://` (model key), or a file path.	yes (user input)
`format`	Source format: `auto` or `onnx`. Auto-detects from the `.onnx` file extension.	yes (user input)
`ir_model`	Store key of the built Relax IR Model (kind `tvm-ir`). Set by `tvm+build` on completion.	no (auto-set by build)
`so_model`	Store key of the compiled Model (kind `tvm-so`). Set by `tvm+compile` on completion.	no (auto-set by compile)

If your ONNX is stored on MinIO via the SDK, use `store://my-project/model/yolov8/source:abc123` instead of `s3://`.

6.2 Step 2: Run tvm+build (source to Relax IR)

Create a `tvm+build` run to convert ONNX to TVM Relax IR. This step parses the model and produces an intermediate representation that is independent of hardware:

```yaml
kind: tvm+build
metadata:
  name: yolov8n-build
  project: my-project
spec:
  function: tvm://my-project/yolov8n:abc123
  image: ~   # uses default builder image: ghcr.io/scc-digitalhub/tvm-toolkit:0.25
  simplify: true
  sanitize_input_names: true
  resources:
    cpu: "2"
    mem: "4Gi"
```

Task spec fields for `tvm+build`:

Field	Type	Used by	Effect
`image`	string	ONNX	Override builder image (default: `runtime.tvm.builders[<format>]`).
`simplify`	bool	ONNX	Run `onnxsim.simplify` to reduce graph complexity.
`target_opset`	int	ONNX	Upgrade/downgrade the ONNX opset before conversion.
`opset_override`	int	ONNX	Override the opset passed to `from_onnx`.
`strict_shape_inference`	bool	ONNX	Strict mode during ONNX shape inference.
`data_prop`	bool	ONNX	Enable data propagation during shape inference.
`keep_params_in_input`	bool	ONNX	Keep weights as graph inputs instead of folding them (produces `params.bin`).
`sanitize_input_names`	bool	ONNX	Rewrite tensor names to valid Relax identifiers.

What happens inside the pod:

The `tvm+build` job runs in a K8s Job on the `tvm-toolkit` image. The `entrypoint.sh` script orchestrates:

1. Init container downloads the source model (`s3://bucket/path/yolov8n.onnx`) into `${TVM_INPUT_DIR}` (typically `/shared/input`).
2. `entrypoint.sh` reads `TVM_TASK_KIND=tvm+build` and translates task fields to CLI arguments for `builder_onnx.py`.
3. `builder_onnx.py` runs (excerpt from the actual script): - `onnx.load` the model - Optionally run shape inference and `onnxsim.simplify` - Call `tvm.relax.frontend.onnx.from_onnx(model, ...)` to produce the Relax `IRModule` - Extract input/output tensor specs from the ONNX graph - Write `model.relax.json` (canonical, round-trip safe) and `metadata.json` to the output folder
4. `_dh_publish.py` calls `digitalhub_sdk.dh.log_tvm_ir(...)` to: - Create a Model entity of kind `tvm-ir` with the extracted signatures - Upload `model.relax.json`, `model.relax.ir` (debug dump), `metadata.json` to S3 - Write the Model key to `run.status.outputs.ir_module`
5. `TvmRuntime.onBuildComplete()` reads `run.status.outputs.ir_module` and writes it to `function.spec.ir_model`.

Example output (on completion):

- `function.spec.ir_model` = `store://my-project/model/tvm-relax-ir/yolov8n-ir/run-id-123`
- S3 artifacts at `s3://digitalhub/my-project/model/tvm-relax-ir/yolov8n-ir/run-id-123/`:
- `model.relax.json` (the IR, JSON-serialized)
- `model.relax.ir` (Relax IR text dump)
- `metadata.json` (input/output signatures, source_format, opset, etc.)

6.3 Step 3: Run tvm+compile (Relax IR to model.so)

Once the IR is built, compile it to a native shared library for the target hardware:

```yaml
kind: tvm+compile
metadata:
  name: yolov8n-compile-cpu
  project: my-project
spec:
  function: tvm://my-project/yolov8n:abc123
  target_architecture: cpu
  opt_level: 3
  resources:
    cpu: "4"
    mem: "8Gi"
```

Compile is memory-hungry (especially for large models); the `8Gi` resource request prevents out-of-memory kills.

Task spec fields for `tvm+compile`:

Field	Type	Default	Effect
`model_path`	string	→ `function.spec.ir_model`	Explicit `store://` IR Model key to compile (overrides auto-fetch).
`target_architecture`	enum	`cpu`	Target: `cpu` (generic host, LLVM baseline), `x86` (x86-64-v2), `arm64` (aarch64, cross-compiled with the toolkit's aarch64 g++). Each expands to a full `tvm.target.Target` string. `llvm` still accepted as a legacy alias for `cpu`. CPU/LLVM only: cuda was removed because no GPU serving path exists.
`opt_level`	int	3	TVM optimization level 0-3.
`cross_cc`	string	,	Cross-compiler for target (e.g. `aarch64-linux-gnu-g++` for ARM).
`exec_mode`	string	`bytecode`	Relax VM mode: `bytecode` or `compiled`.
`relax_pipeline`	string	`default`	Named Relax pass pipeline.
`tir_pipeline`	string	`default`	Named TIR pass pipeline.
`system_lib`	bool	false	Build as system library (advanced).
`params_path`	string	auto	Explicit `params.bin` from a `keep_params_in_input` build; otherwise auto-detected.
`tag`	string	`so`	Free-form tag appended to the Model name (`yolov8n-<tag>`).
`image`	string	`runtime.tvm.compiler`	Override compiler image.

The field name is `target_architecture`, not `target`: The console form breaks if a field is literally named `target`, so the schema uses `target_architecture` instead. The enum value expands to the full TVM target string:

```
cpu           -> "llvm"    (llvm accepted as a legacy alias)
x86           -> {"kind":"llvm","mcpu":"x86-64-v2"}
arm64         -> {"kind":"llvm","mtriple":"aarch64-linux-gnu"}
```

What happens inside the pod:

1. Init container downloads the IR directory from `function.spec.ir_model` (store:// resolved to s3://) into `${TVM_INPUT_DIR}`.
2. `entrypoint.sh` reads `TVM_TASK_KIND=tvm+compile` and builds CLI args for `compiler.py`.

3. `compiler.py` (excerpt): - Load the IR: prefer `model.relax.json` (canonical), fall back to `model.relax.ir` (Relax IR text) - Load `params.bin` if present (produced by a `keep_params_in_input` build) - Call `tvm.target.Target(target_string)` to parse the target - Call `tvm.relax.build(ir_module, target)` to lower Relax to TIR and emit machine code - Call `export_library(model.so, ...)` to write the native shared library - Write metadata (target, opt_level, etc.) to `metadata.json`
4. `_dh_publish.py` calls `digitalhub_sdk.dh.log_tvm_so(...)` to: - Create a Model entity of kind `tvm-so` with the target and compile settings - Optionally add a CONSUMES relationship to the source IR Model (if `TVM_SOURCE_IR_KEY` is set) - Upload `model.so` and `metadata.json` to S3 - Write the Model key to `run.status.outputs.compiled_so`
5. `TvmRuntime.onCompileComplete()` reads the key and writes it to `function.spec.so_model`.

Example output:

- `function.spec.so_model` = `store://my-project/model/tvm-compiled-so/yolov8n-so/run-id-456`
- S3 artifacts at `s3://digitalhub/my-project/model/tvm-compiled-so/yolov8n-so/run-id-456/`:
- `model.so` (the compiled shared library)
- `metadata.json` (target string, opt_level, manifest)

6.4 Step 4: Run tvm+serve (deploy the compiled model)

Deploy the compiled model behind a model-centric serving stack. An init container downloads `model.so` at pod startup; the generic Rust tvm-serve image loads it and exposes Open Inference v2:

```yaml
kind: tvm+serve
metadata:
  name: yolov8n-serve
  project: my-project
spec:
  function: tvm://my-project/yolov8n:abc123
  served_name: yolov8n
  replicas: 1
  workers: 2          # per-pod serve workers (concurrent inferences)
  service_type: ClusterIP
  resources:
    cpu: "4"
    mem: "2Gi"
```

Task spec fields for `tvm+serve`:

Field	Type	Default	Effect
`model_path`	string	→ `function.spec.so_model`	Explicit `store://` compiled Model key to serve.
`served_name`	string	function name (cleaned)	Model name exposed at `/v2/models/<served_name>`.
`image`	string	`runtime.tvm.serve`	Override serve image (default: Rust tvm-runtime-rust:0.25).
`replicas`	int	,	Deployment replica count (horizontal scaling, one pod each).
`workers`	int	1	Per-pod serve workers, wired to `TVM_SERVE_WORKERS`; each loads its own model copy for up to N concurrent inferences.
`service_type`	enum	`ClusterIP`	`ClusterIP`, `NodePort`, or `LoadBalancer`.
`service_name`	string	,	Extra Service alias `<funcName>-<service_name>`.

What happens inside the pod:

1. Kubernetes Deployment is created with `replicas` pods.
2. Init container downloads the `tvm-so` Model's S3 folder into `${TVM_MODEL_DIR}` (typically `/shared/model`).
3. Pod environment is set: - `TVM_MODEL_DIR=/shared/model` - `TVM_MODEL_NAME=yolov8n` - `TVM_SERVE_WORKERS=<workers>` (from `task.workers`, default 1)
4. The serve image's ENTRYPOINT (no command/args overridden by the runner) starts the tvm-serve server: - Loads `/shared/model/model.so` and reads `/shared/model/metadata.json` - Runs the Relax VM - Exposes Open Inference v2:
   - REST on port `8080` (HTTP)
   - gRPC on port `9000` (gRPC)
5. A Kubernetes Service is created (type `ClusterIP`, or `NodePort` / `LoadBalancer` if requested) that routes traffic to the Deployment.

Serve image contract: Any image can be used as long as it respects: - Read `TVM_MODEL_DIR` (a folder with `model.so` + `metadata.json`) - Read `TVM_MODEL_NAME` (the served model name) - Expose Open Inference v2 on ports `8080` (REST) and `9000` (gRPC) - Start automatically from ENTRYPOINT (no command/args injected)

The default image is `ghcr.io/scc-digitalhub/tvm-runtime-rust:0.25` (a native Rust server). Alternatives such as a Go runtime can be swapped via the `image` field or the `RUNTIME_TVM_SERVE` environment variable.

6.5 Step 5: Call inference via REST

Once `tvm+serve:run` is in state `RUNNING`, the service is ready. Query it using the Open Inference v2 API.

REST request (OpenInference v2, JSON):

For a YOLOv8 model expecting input shape `[1, 3, 640, 640]` (batch, RGB channels, height, width):

```
# Get the service endpoint (e.g., from kubectl port-forward or LoadBalancer)
# Assume it's at http://localhost:8080

curl -X POST http://localhost:8080/v2/models/yolov8n/infer \
  -H "Content-Type: application/json" \
  -d '{
    "id": "req-1",
    "inputs": [
      {
        "name": "images",
        "shape": [1, 3, 640, 640],
        "datatype": "FP32",
        "data": [0.5, 0.6, ..., 0.7]
      }
    ]
  }'
```

Response (excerpt):

```
{
  "model_name": "yolov8n",
  "outputs": [
    {
      "name": "output0",
      "shape": [1, 84, 8400],
      "datatype": "FP32",
      "data": [0.1, 0.2, ..., 0.9]
    }
  ],
  "parameters": {
    "inference_time_ms": 42
  }
}
```

The output shape `[1, 84, 8400]` is typical for YOLOv8 nano: `84 = 4 (box) + 80 (COCO classes)` and `8400 = 80*80 + 40*40 + 20*20` (three detection scales).

6.6 Step 6: Call inference via gRPC

For gRPC, use the KServe v2 model serving protocol (Open Inference v2 over gRPC):

```
# Compile the proto (included with test_infer):
grpc_tools_python_protos -I. --python_out=. --grpc_python_out=. grpc_predict_v2.proto

# Python client (excerpt from run_infer.py):
import grpc
import grpc_predict_v2_pb2 as pb
import grpc_predict_v2_pb2_grpc as pbg
import numpy as np

ch = grpc.insecure_channel("localhost:9000")
stub = pbg.GRPCInferenceServiceStub(ch)

# Prepare the request
req = pb.ModelInferRequest(model_name="yolov8n")
t = req.inputs.add()
t.name = "images"
t.datatype = "FP32"
t.shape.extend([1, 3, 640, 640])

# Populate tensor data (as float32 flat array)
img = np.random.randn(1, 3, 640, 640).astype(np.float32)
t.contents.fp32_contents.extend(img.flatten().tolist())

# Call the server
resp = stub.ModelInfer(req, timeout=300)

# Parse output: resp.outputs[0].contents.fp32_contents
output = np.array(resp.outputs[0].contents.fp32_contents).reshape([1, 84, 8400])
```

6.7 Step 7: Decode YOLOv8 detections

The model output is a raw tensor; you must decode it to get bounding boxes and class labels. The `test_infer/run_infer.py` script demonstrates this:

```
cd runtimes/runtime-tvm/test_infer

# Auto-discover the running tvm+serve, download bus.jpg, call inference, decode, save detections
python3 run_infer.py

# Or specify options
python3 run_infer.py --mode grpc --conf 0.25 --image /path/to/photo.jpg
```

What run_infer.py does:

1. Calls CORE API to find the running `tvm+serve:run` and extract the Service name and model name.
2. Sets up `kubectl port-forward` to the Service.

3. Downloads the test image (default: bus.jpg from Ultralytics).
4. Resizes the image to `[640, 640]` using letterboxing (preserves aspect ratio, pads with gray).
5. Normalizes pixel values to `[0, 1]` (or your model's convention).
6. Builds the input tensor (shape `[1, 3, 640, 640]`).
7. Calls the model via REST or gRPC.
8. Decodes the output: - YOLOv8 output is `[1, 84, 8400]` where each of 8400 anchor points has 84 values. - First 4 are box coordinates (cx, cy, w, h) in grid space. - Values 4-83 are class probabilities (80 COCO classes). - Apply sigmoid to confidence and class probs. - Filter by confidence threshold (default 0.25). - Apply Non-Maximum Suppression (NMS) to remove overlapping boxes.
9. Overlays bounding boxes and class labels on the image.
10. Saves the annotated image as `detections.jpg`.

Example invocation (script options):

```
python3 run_infer.py \
  --core http://localhost:8080 \
  --user admin \
  --password admin \
  --project my-project \
  --run <run-id> \
  --mode rest \
  --image bus.jpg \
  --conf 0.3
```

Flag	Default	Meaning
`--mode`	`rest`	`rest` (HTTP) or `grpc` (:9000).
`--project`	`tvm-rust`	CORE project containing the serve run.
`--run`	(auto)	Run ID; omit to use the first RUNNING `tvm+serve:run`.
`--image`	(bus.jpg download)	Local path or URL to the test image.
`--core`	`http://localhost:8080`	CORE API endpoint.
`--user` / `--password`	`admin` / `admin`	CORE credentials (basic auth).
`--input-name` / `--input-shape`	`images` / `1,3,640,640`	Model input; only used if metadata is unavailable (e.g., Go backend).
`--conf`	`0.25`	Confidence threshold for detections.

Pose twin — `test_infer/test_xinet.py`. `test_infer/` also ships `test_xinet.py`, the YOLOv8-**pose** counterpart of `run_infer.py` for the `xinet` model. It reuses the same CORE-API serve discovery, `kubectl port-forward` and OpenInference v2 REST/gRPC path, but its output is `[1, 56, 1029]` (`4` box + `1` person conf + `17×3` COCO keypoints), so instead of detection boxes it decodes **person skeletons**: it resizes the image to `224×224`, runs NMS on the person confidence, reprojects the keypoints to the original resolution and draws the 19 skeleton limbs (one color per limb) on the full-size image (`--out`, `--conf`, `--kp-conf`). See `test_infer/README.md`.

6.8 Console form field mapping

When creating runs in the web console, the UI form fields directly map to spec fields:

Build task (`tvm+build`):

Console field	Task spec field	Java type	Effect
Image	`image`	string	Builder image override
Simplify	`simplify`	bool	Run onnxsim
Target Opset	`target_opset`	int	ONNX opset convert
Opset Override	`opset_override`	int	from_onnx opset override
Strict Shape Inference	`strict_shape_inference`	bool	ONNX shape inference mode
Data Propagation	`data_prop`	bool	Data prop during shape inference
Keep Params in Input	`keep_params_in_input`	bool	Keep weights as graph inputs
Sanitize Input Names	`sanitize_input_names`	bool	Rewrite input names
Resources (CPU, Memory)	`resources.cpu`, `resources.mem`	strings	Pod resource requests

Compile task (`tvm+compile`):

Console field	Task spec field	Java type	Effect
Model Path	`model_path`	string	Explicit IR model (store://)
Target Architecture	`target_architecture`	enum	Target: cpu / x86 / arm64 (CPU/LLVM only; `llvm` legacy alias for cpu)
Optimization Level	`opt_level`	int	TVM opt level 0-3
Cross C++ Compiler	`cross_cc`	string	Cross-compiler binary
Execution Mode	`exec_mode`	string	bytecode or compiled
Relax Pipeline	`relax_pipeline`	string	Relax pass pipeline name
TIR Pipeline	`tir_pipeline`	string	TIR pass pipeline name
System Library	`system_lib`	bool	System-lib module
Params Path	`params_path`	string	Explicit params.bin
Tag	`tag`	string	Model name suffix
Image	`image`	string	Compiler image override
Resources	`resources.cpu`, `resources.mem`	strings	Pod resources (8Gi recommended)

Serve task (`tvm+serve`):

Console field	Task spec field	Java type	Effect
Model Path	`model_path`	string	Explicit .so model (store:///)
Served Name	`served_name`	string	Model name at /v2/models/
Image	`image`	string	Serve image override
Replicas	`replicas`	int	Pod replicas
Workers	`workers`	int	Per-pod serve workers (concurrent inferences)
Service Type	`service_type`	enum	ClusterIP / NodePort / LoadBalancer
Service Name	`service_name`	string	Extra Service alias
Resources	`resources.cpu`, `resources.mem`	strings	Pod resources

---

7. Task and function parameter reference

Function (kind tvm)

The `tvm` function kind accepts a source ML model in ONNX format, plus metadata about its two derived artifacts (IR model and compiled .so). The runtime writes back the IR and .so keys after build and compile complete, so subsequent tasks can locate them automatically.

Field	JSON key	Meaning	Default
model	`model`	Path or store:// key to the source model (ONNX)	required
format	`format`	Source model format: auto, onnx. "auto" infers from the .onnx file extension	`auto`
ir_model	`ir_model`	store:// key of the built Relax IR model (set by tvm+build on completion)	unset
so_model	`so_model`	store:// key of the compiled model.so (set by tvm+compile on completion)	unset

tvm+build task

The `tvm+build` task converts a source model to Relax IR. ONNX preprocessing options are optional; all fields are optional except the base job settings inherited from K8s. The builder image is selected per format from runtime configuration but can be overridden.

Field	JSON key	Meaning	Default	Runn
image	`image`	Override the builder image for this format	from runtime.tvm.builders[format]	n/a
simplify	`simplify`	ONNX: run onnxsim.simplify on the graph before conversion	unset	TVM_S
target_opset	`target_opset`	ONNX: upgrade or downgrade model to this opset before conversion (onnx.version_converter)	unset	TVM_T
opset_override	`opset_override`	ONNX: opset passed to from_onnx, overriding the model's declared opset	unset	TVM_O
strict_shape_inference	`strict_shape_inference`	ONNX: use strict mode during ONNX shape inference	unset	TVM_S
data_prop	`data_prop`	ONNX: enable data propagation during ONNX shape inference	unset	TVM_D
keep_params_in_input	`keep_params_in_input`	Keep model parameters as graph inputs instead of folding them into constants	unset	TVM_K
sanitize_input_names	`sanitize_input_names`	Sanitize input tensor names during conversion to Relax IR	unset	TVM_S

tvm+compile task

The `tvm+compile` task lowers Relax IR to native model.so via a single K8s Job running compiler.py. All fields map to compiler arguments and are optional, with defaults in the runner:

Field	JSON key	Meaning	Default	Runner env var
model_path	`model_path`	Explicit store:// key of the IR model to compile, overriding function.spec.ir_model	function.spec.ir_model	TVM_SOURCE_IR_KEY (if store://)
target_architecture	`target_architecture`	Target hardware: cpu (generic host, LLVM baseline), x86 (x86-64-v2), arm64 (aarch64-linux-gnu; cross_cc defaults to aarch64-linux-gnu-g++). `llvm` is still accepted as a legacy alias for cpu	`cpu`	TVM_TARGET
opt_level	`opt_level`	TVM optimization level 0-3	3 (in runner)	TVM_OPT_LEVEL
exec_mode	`exec_mode`	Relax VM execution mode: bytecode or compiled	bytecode (in runner)	TVM_EXEC_MODE
relax_pipeline	`relax_pipeline`	Named Relax optimization pipeline	default (in runner)	TVM_RELAX_PIPELINE
tir_pipeline	`tir_pipeline`	Named TIR optimization pipeline	default (in runner)	TVM_TIR_PIPELINE
cross_cc	`cross_cc`	Cross C++ compiler for linking when cross-compiling (e.g., aarch64-linux-gnu-g++)	unset	TVM_CROSS_CC
system_lib	`system_lib`	Build a system-lib style module (advanced feature)	false	TVM_SYSTEM_LIB (if true)
params_path	`params_path`	In-pod path to params.bin, passed verbatim as TVM_PARAMS_FILE (store:// / s3:// are **not** resolved for this field, unlike model_path); if unset, a sibling params.bin in the IR dir is auto-detected	unset	TVM_PARAMS_FILE
tag	`tag`	Free-form metadata tag recorded in compiled model metadata and appended to model name	unset	TVM_TAG
image	`image`	Override the compiler image	runtime.tvm.compiler	n/a

tvm+serve task

The `tvm+serve` task deploys a compiled model.so using a model-centric architecture: an init container downloads the .so and metadata.json at pod startup from S3, and a generic tvm-serve image (Rust or Go) exposes OpenInference v2 REST on port 8080 and gRPC on port 9000. The Service type, replicas, and per-pod workers are configurable but ports are fixed.

Field	JSON key	Meaning	Default	Runner env var
model_path	`model_path`	Explicit store:// key of the compiled .so model to serve, overriding function.spec.so_model	function.spec.so_model	n/a
served_name	`served_name`	Model name exposed at /v2/models/	function name (cleaned)	TVM_MODEL_NAME
image	`image`	Override the serve image	runtime.tvm.serve	n/a
replicas	`replicas`	Number of pod replicas in the serving deployment (horizontal scaling)	unset	n/a
workers	`workers`	Number of serve workers per pod; each loads its own model copy, enabling up to N concurrent inferences per pod	1	TVM_SERVE_WORKERS
service_type	`service_type`	Kubernetes Service type: ClusterIP, NodePort, LoadBalancer	`ClusterIP`	n/a
service_name	`service_name`	Custom suffix for additional Service aliases (e.g., funcName-)	unset	n/a